

How EventScore works

Architecture, data model, the rules that look arbitrary and are not, and every bug already found.

- Read before changing scoring or progression
- Full schema and security model
- Six bugs documented so they are not repeated

The code is the source of truth. This is what the code does not say. Several decisions here look like preferences and are load-bearing. Reversing them produces results that are silently wrong rather than obviously broken.

The stack

Piece	What	Why
Framework	Next.js 16, App Router, TypeScript	Server components mean database queries run server-side, so keys never reach the browser. Server actions handle every write.
Database	Supabase Postgres, eu-west-1	Row-level security, auth and file storage in one service. Standard Postgres underneath, so the data is portable.
Hosting	Vercel, function region dub1	Region matters: it must match the database or every query crosses an ocean twice.
Auth	Supabase Auth for staff, custom for judges	Judges have no account at all. See the session section.
Storage	Supabase storage, private bucket	Contestant photographs must never sit on a public URL. Served by signed links valid for an hour.
Styling	Hand-written CSS, one file	No component library. Classes in src/app/globals.css.

Environment variables

Name	Local	Vercel	GitHub	Notes
NEXT_PUBLIC_SUPABASE_URL	Yes	Yes	Yes	Safe to expose.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Yes	Yes	Designed for the browser. Security comes from row-level security, not from hiding it.
SUPABASE_SERVICE_ROLE_KEY	Yes	Yes	No	Bypasses every security rule. Never give it the NEXT_PUBLIC prefix.
CRON_SECRET	Yes	Yes	Yes	Guards the purge endpoint.
APP_URL	No	No	Yes	Where the maintenance workflow sends requests.
SUPABASE_DB_URL	No	No	Yes	Session pooler string for backups. The direct one is IPv6 only and GitHub runners cannot reach it.

The local .env.local file is excluded from git deliberately. Moving hosts means re-entering every variable by hand, and changing one requires a redeploy because they are read at build time.

The data model

profiles → events → rounds → categories, with events also owning contestants and judges. An **entry** joins a contestant to a round. A **score** is one judge marking one entry in one category.

Table	Key columns	Notes
profiles	role, access, status, limits	role is owner or customer. access is full, readonly or disabled.
events	code, winners_count, comments_mode, progression, show_scores, retention_days, logo_url	code is six characters, generated by a trigger on insert.
rounds	position, advance_count, force_closed	advance_count null means this round ends the event.
categories	round_id, name, max_score	weight exists but is always 1; weighting was removed.
contestants	event_id, bib_number, photo_url, status	
entries	round_id, contestant_id	Progression works by absence. No row means they did not advance.
judges	invite_token, pin, position, status	position 1 is the head judge. invite_token is the personal link.
scores	judge_id, entry_id, category_id, value	Unique on the three together. A missing row means zero.
submissions	judge_id, round_id	Presence is what makes a judge's marks count.
entry_comments	judge_id, entry_id, body	
tiebreaks	place, status, tied_entry_ids, winner_entry_id, note	
judge_votes	tiebreak_id, judge_id, chosen_entry_id	
judge_sessions	token, judge_id, event_id, expires_at	Server-side only. No policies.
acceptances	profile_id, document, version, accepted_at	The legal record.
recovery_codes	profile_id, code_hash, used_at	SHA-256 hashes, never the codes.
signin_attempts	identifier, ok, created_at	Rate limiting.
enquiries	name, email, message, handled	Contact form.
audit_log	actor_id, action, target, detail	Administrative actions.

Rules that must not be broken

1. A missing score is a zero

If a judge never touches a slider, no row is written. Both the interface and roundStandings treat that absence as zero.

Earlier the standings required a row for every category and skipped the contestant otherwise, treating them as not having taken part. The visible effect was that seven contestants should have advanced and one did. Anything that recalculates scores must keep the zero behaviour.

2. Judges have no account

A judge opens an invite link carrying a token and enters a PIN. The server creates a row in judge_sessions and puts that token in an httpOnly cookie for twelve hours. Every subsequent judge request validates the token server-side, then queries using the service role key.

This is the only reason the service role key exists in the app. It is also why judge pages are excluded from the proxy: they have their own session mechanism.

3. The owner cannot read event data

There is deliberately no row-level security policy granting an owner access to events, contestants or scores. The database refuses those reads regardless of what the application asks for.

The one exception is customer_event_count, a security-definer function that returns a number and nothing else. This is what makes the claim in the privacy policy true rather than aspirational, and it must survive any migration.

4. Rounds are synchronised

No judge advances until every judge has submitted, so the shortlist is always built from the full panel. The events.progression column allows independent mode, but synchronised is the default and the tested path. The organiser can force a round closed if somebody cannot finish.

5. Rosters freeze once scoring starts

ensureRoster refuses to rebuild a round's line-up once any score exists for it. Without this, a late judge could be shown a different set of contestants from an early one.

6. Ranking is by reach, then by score

In the full results, contestants are ordered first by how far they got, then by their score in that furthest round. Rounds are never averaged together, because a round out of 50 would otherwise swamp one out of 30. The grand total is shown as a record, not used as the ranking key.

Scoring, precisely

For one judge and one contestant in one round:

percentage = mean over categories of (mark / category maximum) × 100
raw marks = sum of marks across categories

A missing mark counts as zero in both. The contestant's round score is the mean across every judge who has **submitted** the round. Judges who have not submitted are excluded entirely, which is why standings are labelled provisional until everyone is in.

The percentage is used for ranking and progression because it makes rounds of different sizes comparable. Raw marks are what appear on screen, because organisers and contestants think in marks.

Tie handling

After the last judge submits, ensureBallot looks for groups of contestants within 0.001 of each other that straddle a place that matters: inside winners_count on a final round, or the advance line otherwise. Ties below that are left alone and ordered by entry number.

One ballot runs at a time, highest place first. Resolution: a single judge decides alone; a clear majority wins; anything else falls to the first judge in the panel order. Every outcome is written to tiebreaks with a human-readable note that appears on the results.

Bugs already found

Symptom	Cause and fix
App unbearably slow, page eventually crashed	A polling useEffect had router in its dependency array. Next.js returns a new router object each render and router.refresh() causes a render, so intervals compounded into a request storm. The router now lives in a ref. Never put router in a dependency array.
Submit button stuck on Submitting forever	The server action called revalidatePath while the client also called router.refresh(). The two race and the transition never settles. Do one or the other, not both.
Submit greyed out despite every category showing a mark	Untouched sliders below the fold counted as incomplete, and the dock was covering the last rows so the judge could not see them.
Contestants not advancing to the next round	roundStandings treated a missing score row as did-not-participate rather than as zero.
Podium place numbers missing for second and third	The number came from an array index that gets reordered for the visual layout. It now comes from the true placing, looked up by identity.
Large empty gap at the top of the homepage	The decorative glow div was in the document flow, pushing everything down by its own height. It needed position absolute.

Offline behaviour

Marks go into localStorage first, keyed by entry and category so repeated changes to the same category only queue once. The screen updates immediately; reaching the server is the queue's problem.

useMarkSync drains the queue roughly 600 milliseconds after the last change, backing off exponentially on failure up to about a minute. It listens for the browser's online event and for the tab becoming visible again. Submitting a round flushes the queue first and refuses if anything is still unsent.

localStorage rather than IndexedDB because the payload is tiny and synchronous access means nothing is lost if the page is closing mid-write.

What this does not solve

A judge cannot sign in with no connection, because the server must verify the PIN and load the contestants. Final standings cannot be produced while devices are offline, because they need every judge's marks in one place. Neither is solvable without becoming an installed application with a local database.

Scheduled work

Workflow	When	What
maintenance.yml	Every two days	Pings Supabase to prevent the free tier sleeping, then calls the purge endpoint.
backup.yml	Nightly at 03:00 UTC	Dumps schema and data via the Supabase CLI, commits both into backups/, keeps thirty days.
schema.yml	Manual	Exports the live schema to db/01-schema.sql. Run after any schema change.

The purge endpoint at /api/cron/purge deletes events past their retention window, photographs included, and is guarded by CRON_SECRET.

Postgres plans a whole query batch before executing it. An ALTER TABLE and an UPDATE referencing the new column must be separate queries, or the second fails on a column that does not exist yet. This catches people out every time.

Known gaps

Gap	Detail
Contestant photo deletion	Deleting an event removes its storage folder. Deleting a single contestant does not, so their photograph persists.
Assist mode	Designed but not built. The owner would request time-limited access and the customer approve it, preserving the content-blind boundary.
Password reset by email	No email provider is configured. Only an owner can reset a password.

Gap	Detail
Independent progression	The column and code path exist but are untested.
Category weighting	The column exists and is always 1. Weighting was deliberately removed.